32.1 Introduction

The boot loader controls initial operation after reset and also provides the tools for programming the flash memory. This could be initial programming of a blank device, erasure and re-programming of a previously programmed device, or programming of the flash memory by the application program in a running system.

32.2 Features

- In-System Programming: In-System programming (ISP) is programming or reprogramming the on-chip flash memory, using the boot loader software and UART0 serial port. This can be done when the part resides in the end-user board.
- In Application Programming: In-Application (IAP) programming is performing erase and write operation on the on-chip flash memory, as directed by the end-user application code.
- Flash signature generation: built-in hardware can generate a signature for a range of flash addresses, or for the entire flash memory.

32.3 Description

The flash boot loader code is executed every time the part is powered on or reset. The loader can execute the ISP command handler or the user application code. A LOW level after reset at pin P2.10 is considered an external hardware request to start the ISP command handler. Assuming that power supply pins are on their nominal levels when the rising edge on $\overline{\text{RESET}}$ pin is generated, it may take up to 3 ms before P2.10 is sampled and the decision on whether to continue with user code or ISP handler is made. If P2.10 is sampled low and the watchdog overflow flag is set, the external hardware request to start the ISP command handler is ignored. If there is no request for the ISP command handler execution (P2.10 is sampled HIGH after reset), a search is made for a valid user program. If a valid user program is found then the execution control is transferred to it. If a valid user program is not found, the auto-baud routine is invoked.

Pin P2.10 is used as a hardware request signal for ISP and therefore requires special attention. Since P2.10 is in high impedance mode after reset, it is important that the user provides external hardware (a pull-up resistor or other device) to put the pin in a defined state. Otherwise unintended entry into ISP mode may occur.

When ISP mode is entered after a power on reset, the IRC and PLL are used to generate the CCLK of 14.748 MHz. The baud rates that can easily be obtained in this case are: 9600 baud, 19200 baud, 38400 baud, 57600 baud, 115200 baud, and 230400 baud. This may not be the case when ISP is invoked by the user application (see Section 32.8.9 “Re-invoke ISP” on page 647).

A hardware flash signature generation capability is built into the flash memory. this feature can be used to create a signature that can then be used to verify flash contents. Details of flash signature generation are in Section 32.10.

32.3.1 Memory map after any reset

When a user program begins execution after reset, the interrupt vectors are set to point to the beginning of flash memory.

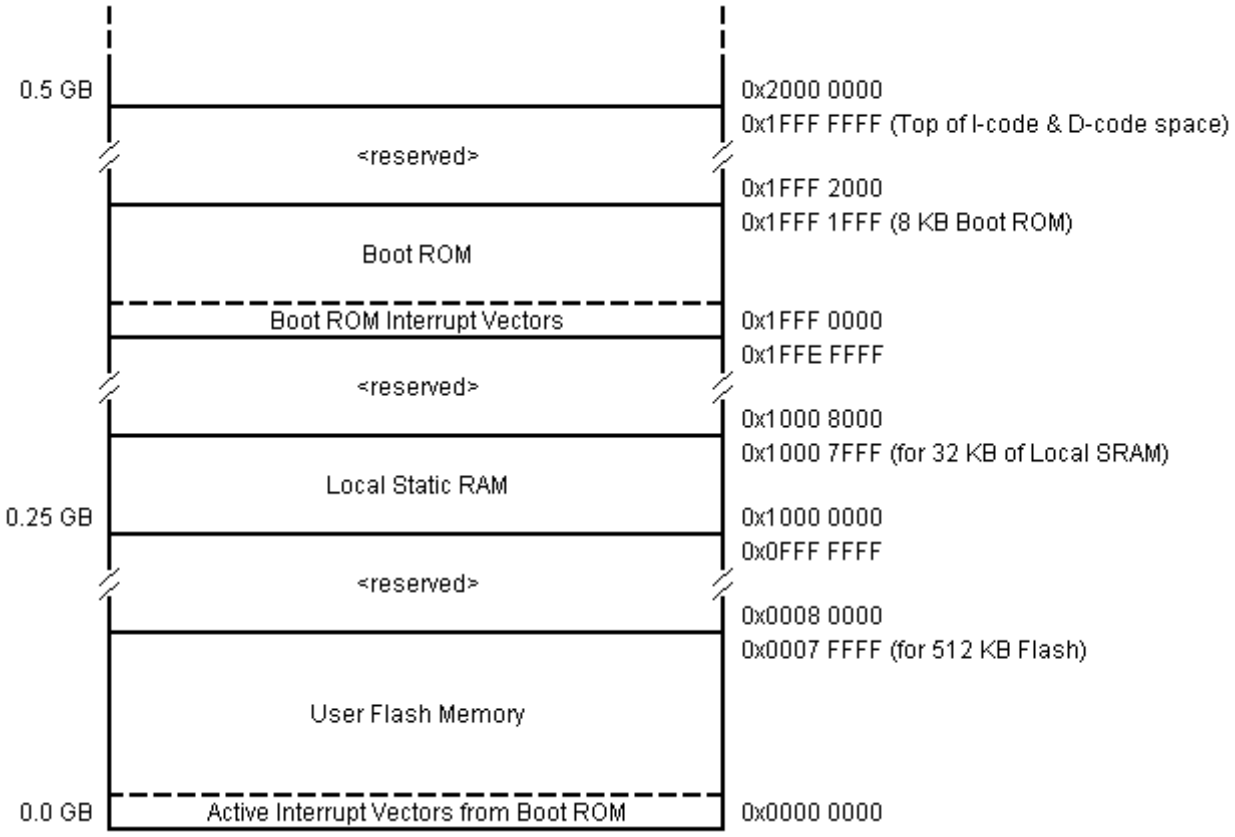


Fig 137. Map of lower memory

32.3.1.1 Criterion for Valid User Code

The reserved Cortex-M3 exception vector location 7 (offset 0x 001C in the vector table) should contain the 2's complement of the check-sum of table entries 0 through 6. This causes the checksum of the first 8 table entries to be 0. The boot loader code checksums the first 8 locations in sector 0 of the flash. If the result is 0, then execution control is transferred to the user code.

If the signature is not valid, the auto-baud routine synchronizes with the host via serial port 0. The host should send a "?" (0x3F) as a synchronization character and wait for a response. The host side serial port settings should be 8 data bits, 1 stop bit and no parity. The auto-baud routine measures the bit time of the received synchronization character in terms of its own frequency and programs the baud rate generator of the serial port. It also

sends an ASCII string ("Synchronized<CR><LF>") to the host. In response to this the host should send the same string ("Synchronized<CR><LF>"). The auto-baud routine looks at the received characters to verify synchronization. If synchronization is verified then "OK<CR><LF>" string is sent to the host. The host should respond by sending the crystal frequency (in kHz) at which the part is running. For example, if the part is running at 10 MHz, the response from the host should be "10000<CR><LF>". "OK<CR><LF>" string is sent to the host after receiving the crystal frequency. If synchronization is not verified then the auto-baud routine waits again for a synchronization character. For auto-baud to work correctly in case of user invoked ISP, the CCLK frequency should be greater than or equal to 10 MHz.

For more details on Reset, PLL and startup/boot code interaction see Section 4.5.1.1 “PLL0 and startup/boot code interaction”.

Once the crystal frequency is received the part is initialized and the ISP command handler is invoked. For safety reasons an "Unlock" command is required before executing the commands resulting in flash erase/write operations and the "Go" command. The rest of the commands can be executed without the unlock command. The Unlock command is required to be executed once per ISP session. The Unlock command is explained in Section 32.7 “ISP commands” on page 634.

32.3.2 Communication protocol

All ISP commands should be sent as single ASCII strings. Strings should be terminated with Carriage Return (CR) and/or Line Feed (LF) control characters. Extra <CR> and <LF> characters are ignored. All ISP responses are sent as <CR><LF> terminated ASCII strings. Data is sent and received in UU-encoded format.

32.3.2.1 ISP command format

"Command Parameter_0 Parameter_1 … Parameter_n<CR><LF>" "Data" (Data only for Write commands).

32.3.2.2 ISP response format

"Return_Code<CR><LF>Response_0<CR><LF>Response_1<CR><LF> … Response_n<CR><LF>" "Data" (Data only for Read commands).

32.3.2.3 ISP data format

The data stream is in UU-encoded format. The UU-encode algorithm converts 3 bytes of binary data in to 4 bytes of printable ASCII character set. It is more efficient than Hex format which converts 1 byte of binary data in to 2 bytes of ASCII hex. The sender should send the check-sum after transmitting 20 UU-encoded lines. The length of any UU-encoded line should not exceed 61 characters (bytes) i.e. it can hold 45 data bytes. The receiver should compare it with the check-sum of the received bytes. If the check-sum matches then the receiver should respond with "OK<CR><LF>" to continue further transmission. If the check-sum does not match the receiver should respond with "RESEND<CR><LF>". In response the sender should retransmit the bytes.

32.3.2.4 ISP flow control

A software XON/XOFF flow control scheme is used to prevent data loss due to buffer overrun. When the data arrives rapidly, the ASCII control character DC3 (0x13) is sent to stop the flow of data. Data flow is resumed by sending the ASCII control character DC1 (0x11). The host should also support the same flow control scheme.

32.3.2.5 ISP command abort

Commands can be aborted by sending the ASCII control character "ESC" (0x1B). This feature is not documented as a command under "ISP Commands" section. Once the escape code is received the ISP command handler waits for a new command.

32.3.2.6 Interrupts during IAP

The on-chip flash memory is not accessible during erase/write operations. When the user application code starts executing the interrupt vectors from the user flash area are active. The user should either disable interrupts, or ensure that user interrupt vectors are active in RAM and that the interrupt handlers reside in RAM, before making a flash erase/write IAP call. The IAP code does not use or disable interrupts.

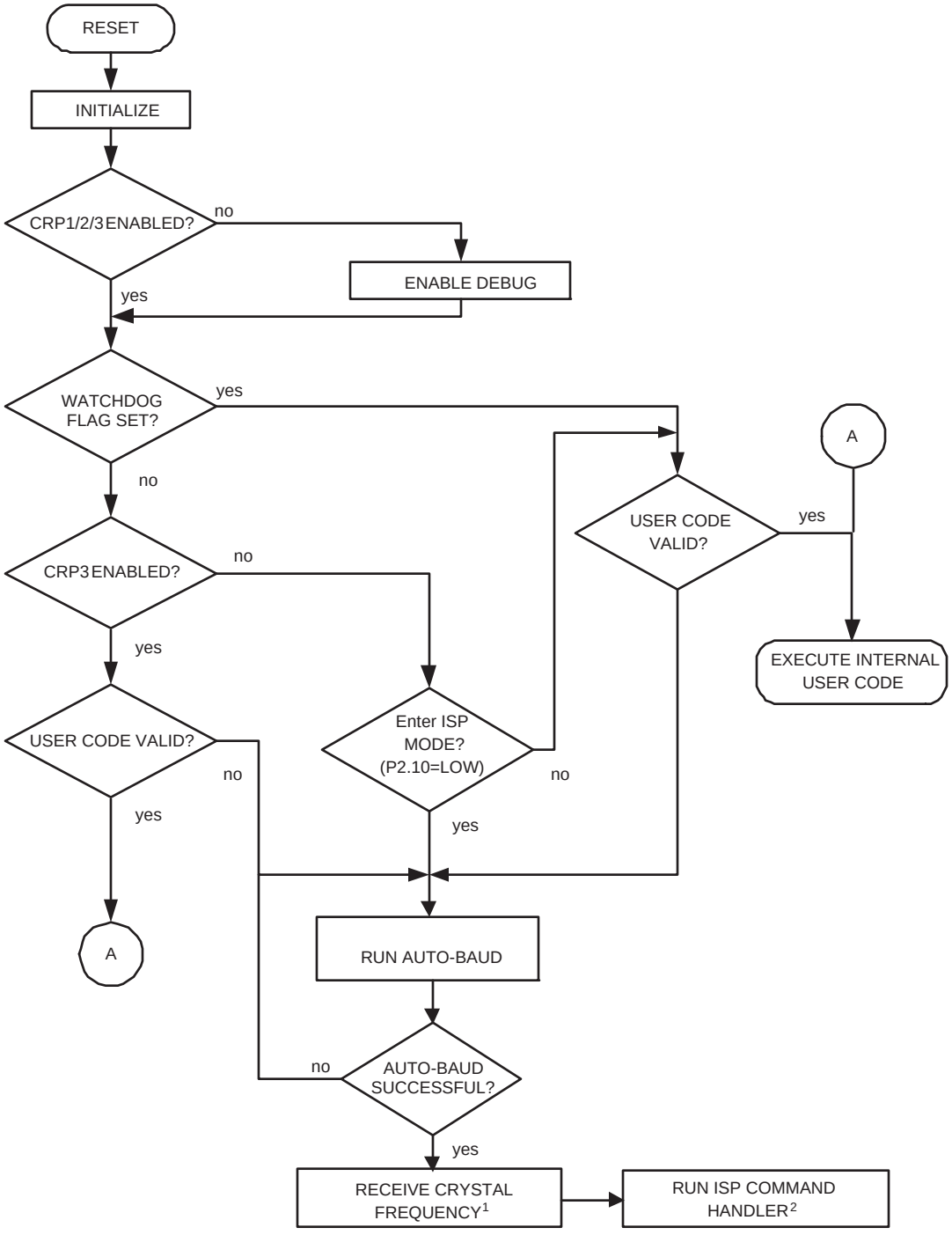
32.3.2.7 RAM used by ISP command handler

ISP commands use on-chip local RAM. from 0x1000 0118 to 0x1000 01FF. The user could use this area, but the contents may be lost upon reset. Flash programming commands use the top 32 bytes of on-chip local RAM. The stack is located at RAM top - 32. The maximum stack usage is 256 bytes and it grows downwards.

32.3.2.8 RAM used by IAP command handler

Flash programming commands use the top 32 bytes of on-chip RAM. The maximum stack usage in the user allocated stack space is 128 bytes and it grows downwards.

32.4 Boot process flowchart



(1) For details on handling the crystal frequency, see Section 32.8.9 “Re-invoke ISP” on page 647

(2) For details on available ISP commands based on the CRP settings see Section 32.6 “Code Read Protection (CRP)”

Fig 138. Boot process flowchart

32.5 Sector numbers

Some IAP and ISP commands operate on "sectors" and specify sector numbers. The following table indicate the correspondence between sector numbers and memory addresses for LPC176x/5x devices containing 32, 64, 128, 256 and 512 kB of flash respectively. IAP and ISP routines are located in the Boot ROM.

Table 568. Sectors in a LPC176x/5x device

Sector Number	Sector Size [kB]	Start Address	End Address	32 kB Part	64 kB Part	128 kB Part	256 kB Part	512 kB Part
0	4	0X0000 0000	0X0000 0FFF	x	x	x	x	x
1	4	0X0000 1000	0X0000 1FFF	x	x	x	x	x
2	4	0X0000 2000	0X0000 2FFF	x	x	x	x	x
3	4	0X0000 3000	0X0000 3FFF	x	x	x	x	x
4	4	0X0000 4000	0X0000 4FFF	x	x	x	x	x
5	4	0X0000 5000	0X0000 5FFF	x	x	x	x	x
6	4	0X0000 6000	0X0000 6FFF	x	x	x	x	x
7	4	0X0000 7000	0X0000 7FFF	x	x	x	x	x
8	4	0x0000 8000	0X0000 8FFF		x	x	x	x
9	4	0x0000 9000	0X0000 9FFF		x	x	x	x
10 (0x0A)	4	0x0000 A000	0X0000 AFFF		x	x	x	x
11 (0x0B)	4	0x0000 B000	0X0000 BFFF		x	x	x	x
12 (0x0C)	4	0x0000 C000	0X0000 CFFF		x	x	x	x
13 (0x0D)	4	0x0000 D000	0X0000 DFFF		x	x	x	x
14 (0X0E)	4	0x0000 E000	0X0000 EFFF		x	x	x	x
15 (0x0F)	4	0x0000 F000	0X0000 FFFF		x	x	x	x
16 (0x10)	32	0x0001 0000	0x0001 7FFF			x	x	x
17 (0x11)	32	0x0001 8000	0x0001 FFFF			x	x	x
18 (0x12)	32	0x0002 0000	0x0002 7FFF				x	x
19 (0x13)	32	0x0002 8000	0x0002 FFFF				x	x
20 (0x14)	32	0x0003 0000	0x0003 7FFF				x	x
21 (0x15)	32	0x0003 8000	0x0003 FFFF				x	x
22 (0x16)	32	0x0004 0000	0x0004 7FFF					x
23 (0x17)	32	0x0004 8000	0x0004 FFFF					x
24 (0x18)	32	0x0005 0000	0x0005 7FFF					x
25 (0x19)	32	0x0005 8000	0x0005 FFFF					x
26 (0x1A)	32	0x0006 0000	0x0006 7FFF					x
27 (0x1B)	32	0x0006 8000	0x0006 FFFF					x
28 (0x1C)	32	0x0007 0000	0x0007 7FFF					x
29 (0x1D)	32	0x0007 8000	0x0007 FFFF					x

32.6 Code Read Protection (CRP)

Code Read Protection is a mechanism that allows user to enable different levels of security in the system so that access to the on-chip flash and use of the ISP can be restricted. When needed, CRP is invoked by programming a specific pattern in flash location at 0x000002FC. IAP commands are not affected by the code read protection.

Important: Any CRP change becomes effective only after the device has gone through a power cycle.

Table 569. Code Read Protection options[1]

Name	Pattern programmed in 0x000002FC	Description
CRP1	0x12345678	Access to chip via the JTAG pins is disabled. This mode allows partial flash update using the following ISP commands and restrictions: • Write to RAM command can not access RAM below 0x10000200. This is due to use of the RAM by the ISP code, see Section 32.3.2.7. • Read Memory command: disabled. • Copy RAM to Flash command: cannot write to Sector 0. • Go command: disabled. • Erase sector(s) command: can erase any individual sector except sector 0 only, or can erase all sectors at once. • Compare command: disabled This mode is useful when CRP is required and flash field updates are needed but all sectors can not be erased. The compare command is disabled, so in the case of partial flash updates the secondary loader should implement a checksum mechanism to verify the integrity of the flash.
CRP2	0x87654321	This is similar to CRP1 with the following additions: • Write to RAM command: disabled. • Copy RAM to Flash: disabled. • Erase command: only allows erase of all sectors.
CRP3	0x43218765	This is similar to CRP2, but ISP entry by pulling P2.10 LOW is disabled if a valid user code is present in flash sector 0. This mode effectively disables ISP override using the P2.10 pin. It is up to the user's application to provide for flash updates by using IAP calls or by invoking ISP with UART0. **Caution: If CRP3 is selected, no future factory testing can be performed on the device.**

[1] CRP is supported by all LPC176x/5x parts with the exception of part LPC1751 with partID 0x2500 1110. Part LPC1751 with partID 0x2500 1118 supports all three CRP levels (see Errata note).

Table 570. Code Read Protection hardware/software interaction

CRP option	User Code Valid	P2.10 pin at reset	JTAG enabled	LPC176x/5x enters ISP mode	partial flash update in ISP mode
None	No	X	Yes	Yes	Yes
	Yes	High	Yes	No	NA
	Yes	Low	Yes	Yes	Yes
CRP1	No	x	No	Yes	Yes
	Yes	High	No	No	NA
	Yes	Low	No	Yes	Yes
CRP2	No	x	No	Yes	No
	Yes	High	No	No	NA
	Yes	Low	No	Yes	No
CRP3	No	x	No	Yes	No
	Yes	x	No	No	NA

If any CRP mode is enabled and access to the chip is allowed via the ISP, an unsupported or restricted ISP command will be terminated with return code CODE_READ_PROTECTION_ENABLED.

32.7 ISP commands

The following commands are accepted by the ISP command handler. Detailed status codes are supported for each command. The command handler sends the return code INVALID_COMMAND when an undefined command is received. Commands and return codes are in ASCII format.

CMD_SUCCESS is sent by ISP command handler only when received ISP command has been completely executed and the new ISP command can be given by the host. Exceptions from this rule are "Set Baud Rate", "Write to RAM", "Read Memory", and "Go" commands.

Table 571. ISP command summary

ISP Command	Usage	Described in
Unlock	U <Unlock Code>	Table 572
Set Baud Rate	B <Baud Rate> <stop bit>	Table 573
Echo	A <setting>	Table 575
Write to RAM	W <start address> <number of bytes>	Table 576
Read Memory	R <address> <number of bytes>	Table 577
Prepare sector(s) for write operation	P <start sector number> <end sector number>	Table 578
Copy RAM to Flash	C <flash address> <RAM address> <number of bytes>	Table 579
Go	G <address> <Mode>	Table 580
Erase sector(s)	E <start sector number> <end sector number>	Table 581
Blank check sector(s)	I <start sector number> <end sector number>	Table 582
Read Part ID	J	Table 583
Read Boot Code version	K	Table 585
Read serial number	N	Table 586
Compare	M <address1> <address2> <number of bytes>	Table 587

32.7.1 Unlock <Unlock code>

Table 572. ISP Unlock command

Command	U
Input	Unlock code: 23130_{10}
Return Code	CMD_SUCCESS | INVALID_CODE | PARAM_ERROR
Description	This command is used to unlock Flash Write, Erase, and Go commands.
Example	"U 23130<CR><LF>" unlocks the Flash Write/Erase & Go commands.

32.7.2 Set Baud Rate <Baud Rate> <stop bit>

Table 573. ISP Set Baud Rate command

Command	B
Input	Baud Rate: 9600 | 19200 | 38400 | 57600 | 115200 | 230400 Stop bit: 1 | 2
Return Code	CMD_SUCCESS | INVALID_BAUD_RATE | INVALID_STOP_BIT | PARAM_ERROR
Description	This command is used to change the baud rate. The new baud rate is effective after the command handler sends the CMD_SUCCESS return code.
Example	"B 57600 1<CR><LF>" sets the serial port to baud rate 57600 bps and 1 stop bit.

Table 574. Correlation between possible ISP baudrates and CCLK frequency (in MHz)

ISP Baudrate .vs. CCLK Frequency	9600	19200	38400	57600	115200	230400
10.0000	+	+	+			
11.0592	+	+		+		
12.2880	+	+	+			
14.7456[1]	+	+	+	+	+	+
15.3600	+					
18.4320	+	+		+		
19.6608	+	+	+			
24.5760	+	+	+			
25.0000	+	+	+			

[1] ISP entry after reset uses the on chip IRC and PLL to run the device at CCLK = 14.748 MHz

32.7.3 Echo <setting>

Table 575. ISP Echo command

Command	A
Input	Setting: ON = 1 | OFF = 0
Return Code	CMD_SUCCESS | PARAM_ERROR
Description	The default setting for echo command is ON. When ON the ISP command handler sends the received serial data back to the host.
Example	"A 0<CR><LF>" turns echo off.

32.7.4 Write to RAM <start address> <number of bytes>

The host should send the data only after receiving the CMD_SUCCESS return code. The host should send the check-sum after transmitting 20 UU-encoded lines. The checksum is generated by adding raw data (before UU-encoding) bytes and is reset after transmitting 20 UU-encoded lines. The length of any UU-encoded line should not exceed 61 characters (bytes) i.e. it can hold 45 data bytes. When the data fits in less than 20 UU-encoded lines then the check-sum should be of the actual number of bytes sent.

The ISP command handler compares it with the check-sum of the received bytes. If the check-sum matches, the ISP command handler responds with "OK<CR><LF>" to continue further transmission. If the check-sum does not match, the ISP command handler responds with "RESEND<CR><LF>". In response the host should retransmit the bytes.

Table 576. ISP Write to RAM command

Command	W
Input	**Start Address:** RAM address where data bytes are to be written. This address should be a word boundary. **Number of Bytes:** Number of bytes to be written. Count should be a multiple of 4
Return Code	CMD_SUCCESS | ADDR_ERROR (Address not on word boundary) | ADDR_NOT_MAPPED | COUNT_ERROR (Byte count is not multiple of 4) | PARAM_ERROR | CODE_READ_PROTECTION_ENABLED
Description	This command is used to download data to RAM. Data should be in UU-encoded format. This command is blocked when code read protection levels CRP2 or CRP3 are enabled.
Example	"W 268435968 4<CR><LF>" writes 4 bytes of data to address 0x1000 0200.

32.7.5 Read Memory <address> <no. of bytes>

The data stream is followed by the command success return code. The check-sum is sent after transmitting 20 UU-encoded lines. The checksum is generated by adding raw data (before UU-encoding) bytes and is reset after transmitting 20 UU-encoded lines. The length of any UU-encoded line should not exceed 61 characters (bytes) i.e. it can hold 45 data bytes. When the data fits in less than 20 UU-encoded lines then the check-sum is of actual number of bytes sent. The host should compare it with the checksum of the received bytes. If the check-sum matches then the host should respond with "OK<CR><LF>" to continue further transmission. If the check-sum does not match then the host should respond with "RESEND<CR><LF>". In response the ISP command handler sends the data again.

Table 577. ISP Read Memory command

Command	R
Input	**Start Address:** Address from where data bytes are to be read. This address should be a word boundary. **Number of Bytes:** Number of bytes to be read. Count should be a multiple of 4.
Return Code	CMD_SUCCESS followed by <actual data (UU-encoded)> | ADDR_ERROR (Address not on word boundary) | ADDR_NOT_MAPPED | COUNT_ERROR (Byte count is not a multiple of 4) | PARAM_ERROR | CODE_READ_PROTECTION_ENABLED
Description	This command is used to read data from RAM or flash memory. This command is blocked when any level of code read protection is enabled.
Example	"R 268435968 4<CR><LF>" reads 4 bytes of data from address 0x1000 0200.

32.7.6 Prepare sector(s) for write operation <start sector number> <end sector number>

This command makes flash write/erase operation a two step process.

Table 578. ISP Prepare sector(s) for write operation command

Command	P
Input	**Start Sector Number** **End Sector Number:** Should be greater than or equal to start sector number.
Return Code	CMD_SUCCESS | BUSY | INVALID_SECTOR | PARAM_ERROR
Description	This command must be executed before executing "Copy RAM to Flash" or "Erase Sector(s)" command. Successful execution of the "Copy RAM to Flash" or "Erase Sector(s)" command causes relevant sectors to be protected again. To prepare a single sector use the same "Start" and "End" sector numbers.
Example	"P 0 0<CR><LF>" prepares the flash sector 0.

32.7.7 Copy RAM to Flash <flash address> <RAM address> <no of bytes>

Table 579. ISP Copy command

Command	C
Input	**Destination Flash Address (DST):** Destination flash address where data bytes are to be written. The destination address should be a 256 byte boundary. **Source RAM Address (SRC):** Source RAM address from which data is to be read. **Number of Bytes:** Number of bytes to be written. Should be 256 | 512 | 1024 | 4096.
Return Code	CMD_SUCCESS | SRC_ADDR_ERROR (Address not on word boundary) | DST_ADDR_ERROR (Address not on correct boundary) | SRC_ADDR_NOT_MAPPED | DST_ADDR_NOT_MAPPED | COUNT_ERROR (Byte count is not 256 | 512 | 1024 | 4096) | SECTOR_NOT_PREPARED_FOR WRITE_OPERATION | BUSY | CMD_LOCKED | PARAM_ERROR | CODE_READ_PROTECTION_ENABLED
Description	This command is used to program the flash memory. The "Prepare Sector(s) for Write Operation" command should precede this command. The affected sectors are automatically protected again once the copy command is successfully executed. This command is blocked when code read protection levels CRP2 or CRP3 are enabled. When code read protection level CRP1 is enabled, individual sectors other than sector 0 can be written.
Example	"C 0 268468224 512<CR><LF>" copies 512 bytes from the RAM address 0x1000 8000 to the flash address 0.

32.7.8 Go <address> <mode>

Table 580. ISP Go command

Command	G
Input	**Address:** Flash or RAM address from which the code execution is to be started. This address should be on a word boundary. **Mode (retained for backward compatibility):** T (Execute program in Thumb Mode) | A (not allowed).
Return Code	CMD_SUCCESS | ADDR_ERROR | ADDR_NOT_MAPPED | CMD_LOCKED | PARAM_ERROR | CODE_READ_PROTECTION_ENABLED
Description	This command is used to execute a program residing in RAM or flash memory. It may not be possible to return to the ISP command handler once this command is successfully executed. This command is blocked when any level of code read protection is enabled.
Example	"G 0 T<CR><LF>" branches to address 0x0000 0000.

When the GO command is used, execution begins at the specified address (assuming it is an executable address) with the device left as it was configured for the ISP code. This means that some things are different than they would be for entering user code directly following a chip reset. Most importantly, the main PLL will be running and connected, configured to generate a CPU clock with a frequency of approximately 14.7456 MHz.

32.7.9 Erase sector(s) <start sector number> <end sector number>

Table 581. ISP Erase sector command

Command	E
Input	**Start Sector Number** **End Sector Number:** Should be greater than or equal to start sector number.
Return Code	CMD_SUCCESS | BUSY | INVALID_SECTOR | SECTOR_NOT_PREPARED_FOR_WRITE_OPERATION | CMD_LOCKED | PARAM_ERROR | CODE_READ_PROTECTION_ENABLED
Description	This command is used to erase one or more sector(s) of on-chip flash memory. This command is blocked when code read protection level CRP3 is enabled. When code read protection level CRP1 is enabled, individual sectors other than sector 0 can be erased. All sectors can be erased at once in CRP1 and CRP2.
Example	"E 2 3<CR><LF>" erases the flash sectors 2 and 3.

32.7.10 Blank check sector(s) <sector number> <end sector number>

Table 582. ISP Blank check sector command

Command	I
Input	**Start Sector Number:** **End Sector Number:** Should be greater than or equal to start sector number.
Return Code	CMD_SUCCESS | SECTOR_NOT_BLANK (followed by <Offset of the first non blank word location> <Contents of non blank word location>) | INVALID_SECTOR | PARAM_ERROR |
Description	This command is used to blank check one or more sectors of on-chip flash memory.
Example	"I 2 3<CR><LF>" blank checks the flash sectors 2 and 3.

32.7.11 Read Part Identification number

Table 583. ISP Read Part Identification command

Command	J
Input	None.
Return Code	CMD_SUCCESS followed by part identification number in ASCII (see Table 584 "LPC176x/5x part identification numbers").
Description	This command is used to read the part identification number. The part identification number maps to a feature subset within a device family. This number will not normally change as a result of technical revisions.

Table 584. LPC176x/5x part identification numbers

Device	ASCII/dec coding	Hex coding
LPC1769	638664503	0x2611 3F37
LPC1768	637615927	0x2601 3F37
LPC1767	637610039	0x2601 2837
LPC1766	637615923	0x2601 3F33
LPC1765	637613875	0x2601 3733
LPC1764	637606178	0x2601 1922
LPC1763	637607987	0x2601 2033
LPC1759	621885239	0x2511 3737
LPC1758	620838711	0x2501 3F37
LPC1756	620828451	0x2501 1723
LPC1754	620828450	0x2501 1722
LPC1752	620761377	0x2500 1121
LPC1751	620761368	0x2500 1118
LPC1751[1]	620761360	0x2500 1110

[1] This part does not support CRP (see Errata note).

32.7.12 Read Boot Code version number

Table 585. ISP Read Boot Code version number command

Command	K
Input	None
Return Code	CMD_SUCCESS followed by 2 bytes of boot code version number in ASCII format. It is to be interpreted as <byte1(Major)>.<byte0(Minor)>.
Description	This command is used to read the boot code version number.

32.7.13 Read device serial number

Table 586. ISP Read device serial number command

Command	N
Input	None.
Return Code	CMD_SUCCESS followed by the device serial number in 4 decimal ASCII groups, each representing a 32-bit value.
Description	This command is used to read the device serial number. The serial number may be used to uniquely identify a single unit among all LPC176x/5x devices.

32.7.14 Compare <address1> <address2> <no of bytes>

Table 587. ISP Compare command

Command	M
Input	**Address1 (DST):** Starting flash or RAM address of data bytes to be compared. This address should be a word boundary. **Address2 (SRC):** Starting flash or RAM address of data bytes to be compared. This address should be a word boundary. **Number of Bytes:** Number of bytes to be compared; should be a multiple of 4.
Return Code	CMD_SUCCESS | (Source and destination data are equal) COMPARE_ERROR | (Followed by the offset of first mismatch) COUNT_ERROR (Byte count is not a multiple of 4) | ADDR_ERROR | ADDR_NOT_MAPPED | PARAM_ERROR |
Description	This command is used to compare the memory contents at two locations. This command is blocked when any level of code read protection is enabled.
Example	"M 8192 268435968 4<CR><LF>" compares 4 bytes from the RAM address 0x1000 0200 to the 4 bytes from the flash address 0x2000.

32.7.15 ISP Return Codes

Table 588. ISP Return Codes Summary

Return Code	Mnemonic	Description
0	CMD_SUCCESS	Command is executed successfully. Sent by ISP handler only when command given by the host has been completely and successfully executed.
1	INVALID_COMMAND	Invalid command.
2	SRC_ADDR_ERROR	Source address is not on word boundary.
3	DST_ADDR_ERROR	Destination address is not on a correct boundary.
4	SRC_ADDR_NOT_MAPPED	Source address is not mapped in the memory map. Count value is taken into consideration where applicable.
5	DST_ADDR_NOT_MAPPED	Destination address is not mapped in the memory map. Count value is taken into consideration where applicable.
6	COUNT_ERROR	Byte count is not multiple of 4 or is not a permitted value.
7	INVALID_SECTOR	Sector number is invalid or end sector number is greater than start sector number.
8	SECTOR_NOT_BLANK	Sector is not blank.
9	SECTOR_NOT_PREPARED_FOR_WRITE_OPERATION	Command to prepare sector for write operation was not executed.
10	COMPARE_ERROR	Source and destination data not equal.
11	BUSY	Flash programming hardware interface is busy.
12	PARAM_ERROR	Insufficient number of parameters or invalid parameter.
13	ADDR_ERROR	Address is not on word boundary.
14	ADDR_NOT_MAPPED	Address is not mapped in the memory map. Count value is taken in to consideration where applicable.
15	CMD_LOCKED	Command is locked.
16	INVALID_CODE	Unlock code is invalid.
17	INVALID_BAUD_RATE	Invalid baud rate setting.
18	INVALID_STOP_BIT	Invalid stop bit setting.
19	CODE_READ_PROTECTION_ENABLED	Code read protection enabled.

32.8 IAP commands

For in-application programming the IAP routine should be called with a word pointer in register r0 pointing to memory (RAM) containing command code and parameters. The result from the IAP command is returned in the table pointed to by register r1. The user can reuse the command table for the result by passing the same pointer in registers r0 and r1. The parameter table should be large enough to hold all of the results in case the number of results are greater than the number of parameters. Parameter passing is illustrated in the Figure 139. The number of parameters and results vary according to the IAP command. The maximum number of parameters is 5, passed to the "Copy RAM to Flash" command. The maximum number of results is 4, returned by the "Read device serial number" command. The command handler sends the status code INVALID_COMMAND when an undefined command is received. The IAP routine resides at location 0x1FFF 1FF0.

The IAP function could be called in the following way using C.

Define the IAP location entry point. Bit 0 of the IAP location is set since the Cortex-M3 uses only Thumb mode.

```
#define IAP_LOCATION 0x1FFF1FF1
```

Define data structure or pointers to pass IAP command table and result table to the IAP function:

```
unsigned long command[5];
unsigned long output[5];
```

or

```
unsigned long * command;
unsigned long * output;
command=(unsigned long *) 0x...
output= (unsigned long *) 0x...
```

Define a pointer to function type, which takes two parameters and returns void. Note the IAP returns the result with the base address of the table residing in R1.

```
typedef void (*IAP)(unsigned int [],unsigned int[]);
IAP iap_entry;
```

Setting function pointer:

```
iap_entry=(IAP) IAP_LOCATION;
```

Whenever you wish to call IAP you could use the following statement.

```
iap_entry (command, output);
```

The IAP call could be simplified further by using the symbol definition file feature supported by ARM Linker in ADS (ARM Developer Suite). You could also call the IAP routine using assembly code.

Note that the first entry in the command table is the IAP command, followed by any required command parameters, starting with Param0. The first entry in the output table is the Return Code, followed by any other results, starting with Result0.

As per the ARM specification (The ARM Thumb Procedure Call Standard SWS ESPC 0002 A-05) up to 4 parameters can be passed in the r0, r1, r2 and r3 registers respectively. Additional parameters are passed on the stack. Up to 4 parameters can be returned in the r0, r1, r2 and r3 registers respectively. Additional parameters are returned indirectly via memory. Some of the IAP calls require more than 4 parameters. If the ARM suggested scheme is used for the parameter passing/returning then it might create problems due to difference in the C compiler implementation from different vendors. The suggested parameter passing scheme reduces such risk.

The flash memory is not accessible during a write or erase operation. IAP commands, which results in a flash write/erase operation, use 32 bytes of space in the top portion of the on-chip RAM for execution. The user program should not be use this space if IAP flash programming is permitted in the application.

Table 589. IAP Command Summary

IAP Command	Command Code	Described in
Prepare sector(s) for write operation	50_{10}	Table 590
Copy RAM to Flash	51_{10}	Table 591
Erase sector(s)	52_{10}	Table 592
Blank check sector(s)	53_{10}	Table 593
Read part ID	54_{10}	Table 594
Read Boot Code version	55_{10}	Table 595
Read device serial number	58_{10}	Table 596
Compare	56_{10}	Table 597
Reinvoke ISP	57_{10}	Table 598

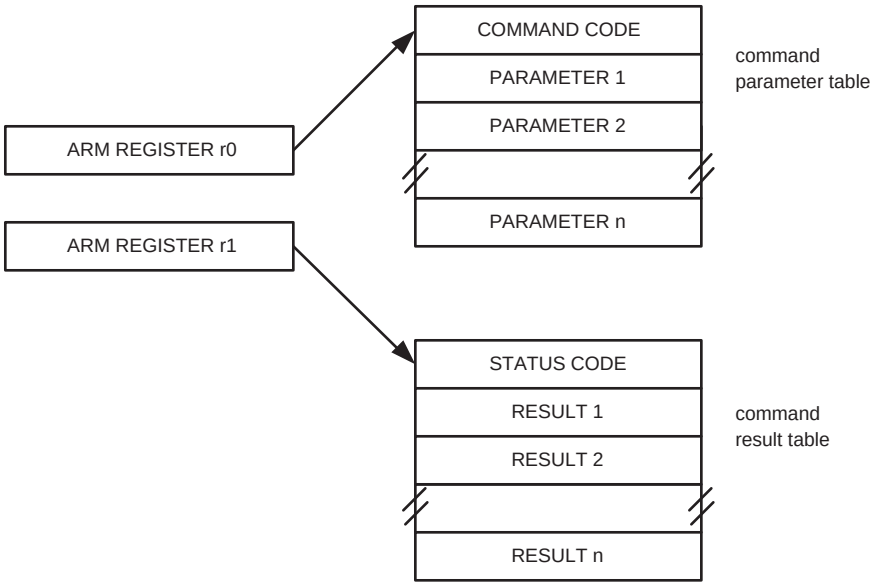


Fig 139. IAP parameter passing

32.8.1 Prepare sector(s) for write operation

This command makes flash write/erase operation a two step process.

Table 590. IAP Prepare sector(s) for write operation command

Command	Prepare sector(s) for write operation
Input	**Command code: $50_{(decimal)}$** **Param0:** Start Sector Number **Param1:** End Sector Number (should be greater than or equal to start sector number).
Return Code	CMD_SUCCESS | BUSY | INVALID_SECTOR
Result	None
Description	This command must be executed before executing "Copy RAM to Flash" or "Erase Sector(s)" command. Successful execution of the "Copy RAM to Flash" or "Erase Sector(s)" command causes relevant sectors to be protected again. To prepare a single sector use the same "Start" and "End" sector numbers.

32.8.2 Copy RAM to Flash

Table 591. IAP Copy RAM to Flash command

Command	Copy RAM to Flash
Input	**Command code: 51$_{(decimal)}$** **Param0(DST):** Destination flash address where data bytes are to be written. This address should be a 256 byte boundary. **Param1(SRC):** Source RAM address from which data bytes are to be read. This address should be a word boundary. **Param2:** Number of bytes to be written. Should be 256 | 512 | 1024 | 4096. **Param3:** CPU Clock Frequency (CCLK) in kHz.
Return Code	CMD_SUCCESS | SRC_ADDR_ERROR (Address not a word boundary) | DST_ADDR_ERROR (Address not on correct boundary) | SRC_ADDR_NOT_MAPPED | DST_ADDR_NOT_MAPPED | COUNT_ERROR (Byte count is not 256 | 512 | 1024 | 4096) | SECTOR_NOT_PREPARED_FOR_WRITE_OPERATION | BUSY |
Result	None
Description	This command is used to program the flash memory. The affected sectors should be prepared first by calling "Prepare Sector for Write Operation" command. The affected sectors are automatically protected again once the copy command is successfully executed.

32.8.3 Erase Sector(s)

Table 592. IAP Erase Sector(s) command

Command	Erase Sector(s)
Input	**Command code: 52$_{(decimal)}$** **Param0:** Start Sector Number **Param1:** End Sector Number (should be greater than or equal to start sector number). **Param2:** CPU Clock Frequency (CCLK) in kHz.
Return Code	CMD_SUCCESS | BUSY | SECTOR_NOT_PREPARED_FOR_WRITE_OPERATION | INVALID_SECTOR
Result	None
Description	This command is used to erase a sector or multiple sectors of on-chip flash memory. To erase a single sector use the same "Start" and "End" sector numbers.

32.8.4 Blank check sector(s)

Table 593. IAP Blank check sector(s) command

Command	Blank check sector(s)
Input	**Command code: 53$_{(decimal)}$** **Param0:** Start Sector Number **Param1:** End Sector Number (should be greater than or equal to start sector number).
Return Code	CMD_SUCCESS | BUSY | SECTOR_NOT_BLANK | INVALID_SECTOR
Result	**Result0:** Offset of the first non blank word location if the Status Code is SECTOR_NOT_BLANK. **Result1:** Contents of non blank word location.
Description	This command is used to blank check a sector or multiple sectors of on-chip flash memory. To blank check a single sector use the same "Start" and "End" sector numbers.

32.8.5 Read part identification number

Table 594. IAP Read part identification number command

Command	Read part identification number
Input	**Command code: 54$_{(decimal)}$** **Parameters:** None
Return Code	CMD_SUCCESS |
Result	**Result0:** Part Identification Number.
Description	This command is used to read the part identification number. The value returned is the hexadecimal version of the part ID. See Table 584 “LPC176x/5x part identification numbers”.

32.8.6 Read Boot Code version number

Table 595. IAP Read Boot Code version number command

Command	Read boot code version number
Input	**Command code: 55$_{(decimal)}$** **Parameters:** None
Return Code	CMD_SUCCESS |
Result	**Result0:** 2 bytes of boot code version number. It is to be interpreted as <byte1(Major)>.<byte0(Minor)>
Description	This command is used to read the boot code version number.

32.8.7 Read device serial number

Table 596. IAP Read device serial number command

Command	Read device serial number
Input	**Command code: 58$_{(decimal)}$**
	Parameters: None
Return Code	CMD_SUCCESS |
Result	**Result0:** First 32-bit word of Device Identification Number (at the lowest address)
	Result1: Second 32-bit word of Device Identification Number
	Result2: Third 32-bit word of Device Identification Number
	Result3: Fourth 32-bit word of Device Identification Number
Description	This command is used to read the device identification number. The serial number may be used to uniquely identify a single unit among all LPC176x/5x devices.

32.8.8 Compare <address1> <address2> <no of bytes>

Table 597. IAP Compare command

Command	Compare
Input	**Command code: 56$_{(decimal)}$**
	Param0(DST): Starting flash or RAM address of data bytes to be compared. This address should be a word boundary.
	Param1(SRC): Starting flash or RAM address of data bytes to be compared. This address should be a word boundary.
	Param2: Number of bytes to be compared; should be a multiple of 4.
Return Code	CMD_SUCCESS |
	COMPARE_ERROR |
	COUNT_ERROR (Byte count is not a multiple of 4) |
	ADDR_ERROR |
	ADDR_NOT_MAPPED
Result	**Result0:** Offset of the first mismatch if the Status Code is COMPARE_ERROR.
Description	This command is used to compare the memory contents at two locations.
	The result may not be correct when the source or destination includes any of the first 64 bytes starting from address zero. The first 64 bytes can be re-mapped to RAM.

32.8.9 Re-invoke ISP

Table 598. Re-invoke ISP

Command	Compare
Input	**Command code: 57$_{(decimal)}$**

Table 598. Re-invoke ISP

Command	Compare
Return Code	None
Result	None.
Description	This command is used to invoke the boot loader in ISP mode. It maps boot vectors, sets PCLK = CCLK / 4, configures UART0 pins Rx and Tx, resets TIMER1 and resets the U0FDR (see Section 14.4.12). This command may be used when a valid user program is present in the internal flash memory and the P2.10 pin is not accessible to force the ISP mode. The command does not disable the PLL hence it is possible to invoke the boot loader when the part is running off the PLL. In such case the ISP utility should pass the CCLK (crystal or PLL output depending on the clock source selection Section 4.4.1) frequency after auto-baud handshake. Another option is to disable the PLL and select the IRC as the clock source before making this IAP call. In this case frequency sent by ISP is ignored and IRC and PLL are used to generate CCLK = 14.748 MHz.

32.8.10 IAP Status Codes

Table 599. IAP Status Codes Summary

Status Code	Mnemonic	Description
0	CMD_SUCCESS	Command is executed successfully.
1	INVALID_COMMAND	Invalid command.
2	SRC_ADDR_ERROR	Source address is not on a word boundary.
3	DST_ADDR_ERROR	Destination address is not on a correct boundary.
4	SRC_ADDR_NOT_MAPPED	Source address is not mapped in the memory map. Count value is taken in to consideration where applicable.
5	DST_ADDR_NOT_MAPPED	Destination address is not mapped in the memory map. Count value is taken in to consideration where applicable.
6	COUNT_ERROR	Byte count is not multiple of 4 or is not a permitted value.
7	INVALID_SECTOR	Sector number is invalid.
8	SECTOR_NOT_BLANK	Sector is not blank.
9	SECTOR_NOT_PREPARED_FOR_WRITE_OPERATION	Command to prepare sector for write operation was not executed.
10	COMPARE_ERROR	Source and destination data is not same.
11	BUSY	Flash programming hardware interface is busy.

32.9 JTAG flash programming interface

Debug tools can write parts of the flash image to the RAM and then execute the IAP call "Copy RAM to Flash" repeatedly with proper offset.

32.10 Flash signature generation

The flash module contains a built-in signature generator. This generator can produce a 128-bit signature from a range of flash memory. A typical usage is to verify the flashed contents against a calculated signature (e.g. during programming).

The address range for generating a signature must be aligned on flash-word boundaries, i.e. 128-bit boundaries. Once started, signature generation completes independently. While signature generation is in progress, the flash memory cannot be accessed for other purposes, and an attempted read will cause a wait state to be asserted until signature generation is complete. Code outside of the flash (e.g. internal RAM) can be executed during signature generation. This can include interrupt services, if the interrupt vector table is re-mapped to memory other than the flash memory. The code that initiates signature generation should also be placed outside of the flash memory.

32.10.1 Register description for signature generation

Table 600. Register overview: FMC (base address 0x4008 4000)

Name	Description	Access	Reset Value	Address	Reference
FMSSTART	Signature start address register	R/W	0	0x4008 4020	Table 601
FMSSTOP	Signature stop-address register	R/W	0	0x4008 4024	Table 602
FMSW0	128-bit signature Word 0	R	-	0x4008 402C	Table 603
FMSW1	128-bit signature Word 1	R	-	0x4008 4030	Table 604
FMSW2	128-bit signature Word 2	R	-	0x4008 4034	Table 605
FMSW3	128-bit signature Word 3	R	-	0x4008 4038	Table 606
FMSTAT	Signature generation status register	R	0	0x4008 4FE0	Section 32.10.1.3
FMSTATCLR	Signature generation status clear register	W	-	0x4008 4FE8	Section 32.10.1.4

32.10.1.1 Signature generation address and control registers

These registers control automatic signature generation. A signature can be generated for any part of the flash memory contents. The address range to be used for generation is defined by writing the start address to the signature start address register (FMSSTART) and the stop address to the signature stop address register (FMSSTOP. The start and stop addresses must be aligned to 128-bit boundaries and can be derived by dividing the byte address by 16.

Signature generation is started by setting the SIG_START bit in the FMSSTOP register. Setting the SIG_START bit is typically combined with the signature stop address in a single write.

Table 601 and Table 602 show the bit assignments in the FMSSTART and FMSSTOP registers respectively.

Table 601. Flash Module Signature Start register (FMSSTART - 0x4008 4020) bit description

Bit	Symbol	Description	Reset Value
31:17	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
16:0	START	Signature generation start address (corresponds to AHB byte address bits[20:4]).	0

Table 602. Flash Module Signature Stop register (FMSSTOP - 0x4008 4024) bit description

Bit	Symbol	Value	Description	Reset Value
31:18	-		Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
17	SIG_START		Start control bit for signature generation.	0
		0	Signature generation is stopped	
		1	Initiate signature generation	
16:0	STOP		BIST stop address divided by 16 (corresponds to AHB byte address [20:4]).	0

32.10.1.2 Signature generation result registers

The signature generation result registers return the flash signature produced by the embedded signature generator. The 128-bit signature is reflected by the four registers FMSW0, FMSW1, FMSW2 and FMSW3.

The generated flash signature can be used to verify the flash memory contents. The generated signature can be compared with an expected signature and thus makes saves time and code space. The method for generating the signature is described in Section 32.10.2.

Table 606 show bit assignment of the FMSW0 and FMSW1, FMSW2, FMSW3 registers respectively.

Table 603. FMSW0 register bit description (FMSW0, address: 0x4008 402C)

Bit	Symbol	Description	Reset Value
31:0	SW0[31:0]	Word 0 of 128-bit signature (bits 31 to 0).	-

Table 604. FMSW1 register bit description (FMSW1, address: 0x4008 4030)

Bit	Symbol	Description	Reset Value
31:0	SW1[63:32]	Word 1 of 128-bit signature (bits 63 to 32).	-

Table 605. FMSW2 register bit description (FMSW2, address: 0x4008 4034)

Bit	Symbol	Description	Reset Value
31:0	SW2[95:64]	Word 2 of 128-bit signature (bits 95 to 64).	-

Table 606. FMSW3 register bit description (FMSW3, address: 0x4008 4038)

Bit	Symbol	Description	Reset Value
31:0	SW3[127:96]	Word 3 of 128-bit signature (bits 127 to 96).	-

32.10.1.3 Flash Module Status register (FMSTAT - 0x0x4008 4FE0)

The read-only FMSTAT register provides a means of determining when signature generation has completed. Completion of signature generation can be checked by polling the SIG_DONE bit in FMSTAT. SIG_DONE should be cleared via the FMSTATCLR register before starting a signature generation operation, otherwise the status might indicate completion of a previous operation.

Table 607. Flash module Status register (FMSTAT - 0x4008 4FE0) bit description

Bit	Symbol	Description	Reset Value
31:2	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
2	SIG_DONE	When 1, a previously started signature generation has completed. See FMSTATCLR register description for clearing this flag.	0
1:0	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

32.10.1.4 Flash Module Status Clear register (FMSTATCLR - 0x0x4008 4FE8)

The FMSTATCLR register is used to clear the signature generation completion flag.

Table 608. Flash Module Status Clear register (FMSTATCLR - 0x0x4008 4FE8) bit description

Bit	Symbol	Description	Reset Value
31:2	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
2	SIG_DONE_CLR	Writing a 1 to this bits clears the signature generation completion flag (SIG_DONE) in the FMSTAT register.	0
1:0	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

32.10.2 Algorithm and procedure for signature generation

Signature generation

A signature can be generated for any part of the flash contents. The address range to be used for signature generation is defined by writing the start address to the FMSSTART register, and the stop address to the FMSSTOP register.

The signature generation is started by writing a '1' to FMSSTOP.MISR_START. Starting the signature generation is typically combined with defining the stop address, which is done in another field FMSSTOP.FMSSTOP of the same register.

The time that the signature generation takes is proportional to the address range for which the signature is generated. Reading of the flash memory for signature generation uses a self-timed read mechanism and does not depend on any configurable timing settings for the flash. A safe estimation for the duration of the signature generation is:

Duration = int((60 / tcy) + 3) x (FMSSTOP - FMSSTART + 1)

When signature generation is triggered via software, the duration is in AHB clock cycles, and tcy is the time in ns for one AHB clock. The SIG_DONE bit in FMSTAT can be polled by software to determine when signature generation is complete.

If signature generation is triggered via JTAG, the duration is in JTAG tck cycles, and tcy is the time in ns for one JTAG clock. Polling the SIG_DONE bit in FMSTAT is not possible in this case.

After signature generation, a 128-bit signature can be read from the FMSW0 to FMSW3 registers. The 128-bit signature reflects the corrected data read from the flash. The 128-bit signature reflects flash parity bits and check bit values.

Content verification

The signature as it is read from the FMSW0 to FMSW3 registers must be equal to the reference signature. The algorithms to derive the reference signature is given in Figure 140.

```
sign = 0
FOR address = FMSTART.FMSTART TO FMSTOP.FMSTOP
{
        FOR i = 0 TO 126
                nextSign[i] = f_Q[address][i] XOR sign[i+1]
                nextSign[127] = f_Q[address][127] XOR sign[0] XOR sign[2] XOR
                        sign[27] XOR sign[29]
        sign = nextSign
}
signature128 = sign
```

Fig 140. Algorithm for generating a 128 bit signature